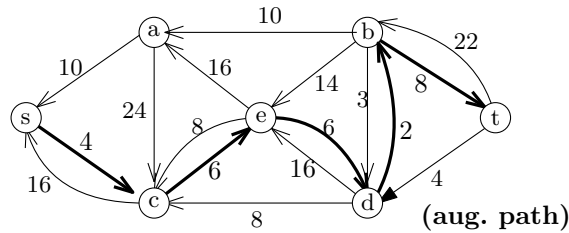


CS 473, Spring 2023

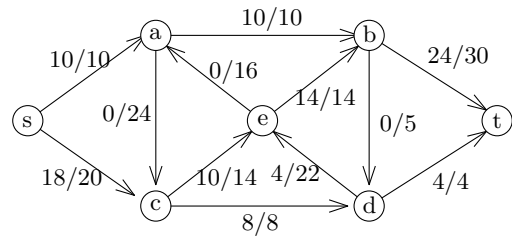
Midterm 2 Solutions

1. (a)

residual graph G_f

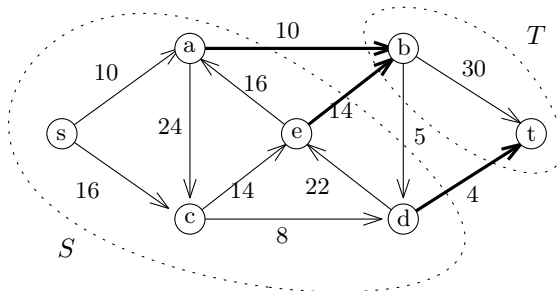


new flow f



flow value $10 + 18 = 28$

an (s, t) -cut with capacity $10 + 14 + 4 = 28$



The new flow is optimal by the max-flow/min-cut theorem, since value of the flow is equal to the capacity of some (s, t) -cut.

(b) (Note: As announced during the exam, there is a typo; it should read “with error probability $3/4$ ”.)

No. Any one of the following explanations will do:

- We can’t assume that the input number is random.
- There could be an input number on which the algorithm is always wrong (so error probability is 1!).
- It’s not told that the algorithm uses a random number generator, or makes random choices.

(c) Run A; then use V to verify the answer; if correct, return answer, else repeat.

Since the probability of success is at least 0.9, the expected number of repetitions is at most $1/0.9 = O(1)$. (The expected number is at most $1 + 0.1 + 0.1^2 + 0.1^3 + \dots$, which converges to a constant.) Thus, the total expected running time is $O(T_A(n) + T_V(n))$.

- (d) True. In each iteration, the size of the current matching increases by 1. Since the size of the matching is at most n (actually, $n/2$), the number of iterations is at most n .

2. (a) *Pseudocode.* (We assume that $\Sigma = \{0, \dots, \sigma - 1\}$.)

0. $x = \text{random}(0, p - 1)$ (where p is a fixed prime)
1. $A_0 = (a_1x^{\ell-1} + a_2x^{\ell-2} + \dots + a_\ell) \bmod p$
2. for $i = 0$ to $n - \ell - 1$ do
3. $A_{i+1} = (A_ix + a_{i+\ell+1} - a_{i+1}x^\ell) \bmod p$
4. $B_0 = (b_1x^{\ell-1} + b_2x^{\ell-2} + \dots + b_\ell) \bmod p$
5. for $j = 0$ to $n - \ell - 1$ do
6. $B_{j+1} = (B_jx + b_{j+\ell+1} - b_{j+1}x^\ell) \bmod p$
7. if there is some i, j with $A_i = B_j$ then
8. if $a_{i+1} \dots a_{i+\ell} = b_{j+1} \dots b_{j+\ell}$ then return yes
9. else return answer by brute force
10. else return no

The algorithm is always correct (Las Vegas): if no i, j exist in line 6, the answer is no, since in order for $a_{i+1} \dots a_{i+\ell} = b_{j+1} \dots b_{j+\ell}$, their fingerprints must match, i.e., $A_i = B_j$. If the test in line 7 succeeds, we have verified that the answer is yes. Otherwise, we run brute force in line 9.

Analysis. We can first precompute $1, x, x^2, \dots, x^\ell$ with $\ell - 1$ multiplications. Afterwards, lines 1 and 4 take $O(\ell)$ arithmetic operations, and lines 3 and 6 takes $O(1)$ arithmetic operations. So, the total number of arithmetic operations is $O(n)$.

Line 7 requires further details, since trivially it would take quadratic time. One way is to store the set $S = \{A_0, \dots, A_{n-\ell}\}$ in a dictionary, and then we can search for b_j in S by a membership query. By the hashing methods from class, the preprocessing time is $O(n)$, and the expected query time is $O(1)$. So, line 7 takes $O(n)$ expected time.

The test in line 8 takes $O(\ell)$ time.

For line 9, a brute-force algorithm takes $O(n^2)$ time, but we argue that the probability that this step is performed is small: Let E_{ij} denote the event that $F_x(a_{i+1} \dots a_{i+\ell}) = F_x(b_{j+1} \dots b_{j+\ell})$ for a given i, j with $a_{i+1} \dots a_{i+\ell} \neq b_{j+1} \dots b_{j+\ell}$. According to a result from class (the ‘‘Alice–Bob’’ subproblem), $\Pr(E_{ij}) \leq (\ell - 1)/p$ (assuming $p \geq \ell - 1$ and $p \geq \sigma$). Thus, the probability that line 7 is executed is (by the union bound) at most

$$\Pr\left(\bigcup_{i=0}^{n-\ell} \bigcup_{j=0}^{n-\ell} E_{ij}\right) \leq \sum_{i=0}^{n-\ell} \sum_{j=0}^{n-\ell} \frac{\ell - 1}{p} \leq (n - \ell + 1)^2 \cdot \frac{\ell - 1}{p} \leq \frac{n^3}{p}.$$

By choosing a prime p near $\max\{n^4, \sigma\}$, for example, this probability is kept below $1/n$. The expected runtime would then be at most $O(n \cdot 1 + n^2 \cdot (1/n)) = O(n)$.

[Note that arithmetic operations are performed only for $\log p = O(\log(n\sigma))$ -bit numbers.]

- (b) We just use binary search to find the largest ℓ for which the algorithm in (a) returns yes. The binary search requires $O(\log n)$ calls to the algorithm in (a). So, the total expected running time is $O(n \log n)$.

3. Pseudocode.

1. repeat d times {
2. form a random subset S where each element is in S with probability $1/2$
3. if the weight of S is not equal to $|S|$ then return “yes”
- }
4. return “no”

The number of weighings is $O(d)$, which is $O(1)$ as we will set d to be a constant later.

Analysis. If there are no fake coins, then the algorithm correctly returns “no”. Suppose there is one or two fake coins. Fix one iteration of the algorithm. If there is exactly one fake coin i , then

$$\Pr\{\text{line 3 returns “yes”}\} = \Pr\{i \in S\} = 1/2.$$

If there are two fake coins i and j , then

$$\begin{aligned} \Pr\{\text{line 3 returns “yes”}\} &= \Pr\{(i \in S \text{ and } j \notin S) \text{ or } (i \notin S \text{ and } j \in S)\} \\ &= \Pr\{i \in S \text{ and } j \notin S\} + \Pr\{i \notin S \text{ and } j \in S\} \quad (\text{by disjointness}) \\ &= (1/2) \cdot (1/2) + (1/2) \cdot (1/2) \quad (\text{by independence}) \\ &= 1/2. \end{aligned}$$

So, the probability that the algorithm errs is at most $1/2^d$ (by independence). Setting $d = 7$, for example, guarantees error probability less than 0.01.

Bonus. We use the same algorithm, but the analysis is trickier. Suppose there is at least one fake coin i_0 . For each subset S , define a related subset S' which is identical to S , except with one difference: if $i_0 \in S$, then $i_0 \notin S'$, and if $i_0 \notin S$, then $i_0 \in S'$. Observe that we cannot have both the weight of S equal to $|S|$ and the weight of S' equal to $|S'|$ (since i_0 has weight not equal to 1). Observe that S and S' are equally likely. This implies that error probability in one iteration is at most $1/2$.

4. Construction. Define a new directed graph $G' = (V', E')$ as follows:

- $V' = V \cup \{s, t, s', t', s'', t''\}$ for 4 new vertices s, t, s', t', s'', t'' , where s and t are the overall source and sink.
- For each $uv \in E$, add an edge (u, v) with capacity 1.
- For each $u \in S_L$, add an edge (s', u) with capacity 1.
- For each $u \in V_L - S_L$, add an edge (s'', u) with capacity 1.
- For each $v \in S_R$, add an edge (v, t') with capacity 1.
- For each $v \in V_R - S_R$, add an edge (v, t'') with capacity 1.

- Add an edge (s, s') with capacity $|S_L|$.
- Add an edge (s, s'') with capacity $k - |S_L|$.
- Add an edge (t', t) with capacity $|S_R|$.
- Add an edge (t'', t) with capacity $k - |S_R|$.

Run a max flow algorithm on G' , and return yes iff the max flow value is k .

Run time. Let n and m be the number of vertices and edges in G . The new graph G' has $O(n)$ vertices and $O(m)$ edges. By using Ford–Fulkerson (we don't need Edmonds–Karp), the run time is bounded by $O(m|f^*|)$, where the optimal flow value $|f^*|$ is at most k . Thus, the run time is $O(mk)$.

Correctness.

To Show: there exists a matching M in G that uses exactly k edges and covers all vertices in V_L and $V_R \iff$ max flow value in G' is k .

Proof of ($\implies$): Suppose there exists such a matching M in G . Define the following flow f in G' :

- $f(s, s') = |S_L|$, $f(s, s'') = k - |S_L|$, $f(t', t) = |S_R|$, and $f(t'', t) = k - |S_R|$.
- For each $u \in S_L$, $f(s', u) = 1$.
- For each $u \in V_L - S_L$, if u is covered by M , then $f(s'', u) = 1$, else $f(s'', u) = 0$.
- For each $uv \in E$, if $uv \in M$, then $f(u, v) = 1$, else $f(u, v) = 0$.
- For each $v \in S_R$, $f(v, t') = 1$.
- For each $v \in V_R - S_R$, if v is covered by M , then $f(v, t'') = 1$, else $f(v, t'') = 0$.

Flow is conserved at s' (resp. t'), because M covers S_L (resp. S_R). Flow is conserved at s'' (resp. t''), because $|M| = k$. The value of f is $|S_L| + (k - |S_L|) = k$. The value of any flow is at most $|S_L| + (k - |S_L|) = k$. Thus, the max flow value is k .

Proof of ($\impliedby$): Suppose the optimal flow f^* in G' has value k . By the Integrality Theorem, $f^*(u, v) \in \{0, 1\}$ for each $uv \in E$. Define

$$M = \{uv \in E : f^*(u, v) = 1\}.$$

Then M is a matching, because the flow into each $u \in V_L$ (resp. out of each $v \in V_R$) is at most 1. And M must have exactly k edges, because f^* has value k . And M covers S_L , because in order for $|f^*|$ to have value k , the edge (s, s') must be saturated and have capacity $|S_L|$. Similarly, M covers S_R .